

Business Requirements Document (BRD)

AI-Powered Predictive Shopping List App

Document Version	1.0
Date	June 28, 2026
Status	Draft
Prepared For	Internal / Stakeholder Review
Platforms	iOS, Android

1. Executive Summary

This document defines the business and functional requirements for a mobile shopping list application that learns a user's purchasing patterns over time and proactively suggests items the user is likely to need, based on purchase frequency and estimated consumption rate.

Key strategic decision: The prediction engine for Phase 1 (MVP) will run **entirely on-device**. No purchase history or personal usage data will be transmitted to a server for the core prediction feature. This decision drives several requirements below around architecture, privacy, and offline capability. Server-side intelligence (aggregate trend data, cold-start priors) is explicitly deferred to Phase 2 and is documented here only to ensure Phase 1 is built in a way that doesn't block it.

2. Business Objectives

ID	Objective
BO-1	Reduce the time and mental effort users spend creating shopping lists
BO-2	Differentiate from generic checklist apps via predictive, personalized suggestions
BO-3	Build user trust through a strong privacy position ("your data stays on your device")
BO-4	Launch quickly with a lean architecture, while leaving room to add server-side intelligence later without a rebuild

ID	Objective
BO-5	Establish a foundation for future monetization (premium prediction features)

3. Project Scope

3.1 In Scope — Phase 1 (MVP)

- Manual shopping list creation and management
- On-device tracking of purchase history (item, date, quantity)
- On-device statistical model to estimate purchase frequency and consumption rate per item, per user
- Auto-generated shopping suggestions based on the on-device model
- Push notifications for predicted "due soon" items
- Offline-first functionality for all core and predictive features
- iOS and Android native or cross-platform clients
- Basic account creation and multi-device sync of list data (not prediction models — see NFR-6)

3.2 Out of Scope — Deferred to Phase 2+

- Server-side aggregate/cross-user prediction models
- Cold-start priors pulled from server-side data
- Federated learning / shared model improvement across users
- Receipt OCR / scanning
- Grocery delivery integrations
- Voice assistant integrations (Siri/Google Assistant)
- Smartwatch companion apps
- Advanced analytics dashboards (spend trends, waste insights)

Note: Phase 2 items are listed to confirm they are **not excluded by architecture** — see Section 7 (Technical Approach) for how Phase 1 is designed to accommodate them later.

4. Stakeholders

Role	Interest
Product Owner	Overall feature prioritization and roadmap

Role	Interest
Mobile Engineering (iOS/Android)	Implementation of on-device model and app logic
UX/UI Design	List management flow, suggestion presentation, cold-start UX
Data/ML (advisory, Phase 1; active, Phase 2)	On-device model design now; server model design later
Privacy/Legal	Ensuring data handling claims match actual architecture
End Users	Primary beneficiaries; household shoppers

5. Functional Requirements

5.1 Core Shopping List Management

ID	Requirement	Priority
FR-1.1	User can add, edit, and delete shopping list items manually	Must
FR-1.2	User can specify quantity, unit, and category per item	Must
FR-1.3	User can mark items as purchased/checked off	Must
FR-1.4	User can maintain multiple lists (e.g., groceries, household)	Should
FR-1.5	App provides autocomplete from a local product/category reference list	Should
FR-1.6	User can add items via voice input	Could
FR-1.7	User can add items via barcode scan	Could

5.2 AI Prediction Engine (On-Device)

ID	Requirement	Priority
FR-2.1	App logs each purchase event locally (item, timestamp, quantity)	Must
FR-2.2	App computes a rolling/average purchase interval per item, per user, entirely on-device	Must
FR-2.3	App generates "due soon" suggestions when an item's predicted next-purchase date approaches	Must

ID	Requirement	Priority
FR-2.4	User can accept, reject, or snooze a suggestion; this feedback adjusts future predictions for that item	Must
FR-2.5	Model gives more weight to recent purchases than older ones (recency weighting)	Should
FR-2.6	User can mark a date range as "vacation/pause" so it is excluded from pattern calculations	Should
FR-2.7	App clearly indicates to the user when an item has insufficient history for a reliable prediction (cold-start state)	Must
FR-2.8	Prediction engine operates fully offline with no network dependency	Must

5.3 Notifications & Reminders

ID	Requirement	Priority
FR-3.1	App sends a push notification when an item is predicted to be running low	Must
FR-3.2	User can configure notification frequency and quiet hours	Should

5.4 Shopping Mode

ID	Requirement	Priority
FR-4.1	App provides a simplified, large-tap checklist view for in-store use	Should
FR-4.2	List items can be sorted by category/aisle	Could
FR-4.3	All shopping-mode functionality works fully offline	Must

5.5 Account & Sync

ID	Requirement	Priority
FR-5.1	User can create an account (email, Apple, Google sign-in)	Must
FR-5.2	List data syncs across the user's own devices	Should

ID	Requirement	Priority
FR-5.3	Locally-derived prediction models/history are NOT required to sync to a server in Phase 1 (architecture should not prevent this in Phase 2 — see NFR-6)	Must

6. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	On-device prediction computation must complete in under 1 second on a mid-range device, for typical list sizes (under ~200 tracked items)
NFR-2	Privacy	Raw purchase history and item-level data must not leave the user's device in Phase 1
NFR-3	Offline capability	All Phase 1 features (list management, predictions, notifications) must function without network connectivity, except account sign-in and cross-device sync
NFR-4	Platform support	App must run on current and prior major iOS and Android OS versions at time of launch
NFR-5	Data durability	Local data (purchase history, model state) must persist reliably across app updates and device restarts
NFR-6	Extensibility	Architecture must allow server-side signals to be added later as an optional input to the suggestion logic, without requiring a redesign of the on-device data model (see Section 7)
NFR-7	Compliance	Data handling must align with GDPR/CCPA; since Phase 1 keeps personal data on-device, compliance burden is reduced but account/profile data still requires standard handling

7. Technical Approach (Architecture Overview)

7.1 Phase 1 — On-Device Only

[Local purchase event log] → [On-device interval/frequency model] → [Suggestion ranker] → [UI]

- Purchase history and the prediction model are stored and computed entirely on-device (e.g., Core ML on iOS, ML Kit/TensorFlow Lite on Android, or a shared cross-

platform statistical implementation if using React Native/Flutter)

- The "suggestion ranker" component must be built as a discrete, pluggable step — accepting on-device signals now, with a defined (but unused in Phase 1) interface for server-side signals later
- Data schema for purchase history should be versioned from the start to avoid migration issues if server sync is introduced later

7.2 Phase 2 (Reference Only — Not Built in Phase 1)

For planning continuity, three future integration paths were evaluated. **Recommended first step for Phase 2 is Option A.**

Option	Description	Complexity
A. Server-provided cold-start priors	Server periodically supplies generic, anonymized "typical interval per category" data to seed predictions for new users/items before enough personal history exists	Low
B. Server-side batch analysis	Anonymized/aggregated usage signals are uploaded periodically; server computes cross-item correlations or seasonal adjustments and pushes back suggestion boosts	Medium
C. Federated learning	On-device models train locally; only model weight updates (not raw data) are aggregated server-side to improve a shared base model	High

Phase 1 architecture decisions (NFR-6, pluggable suggestion ranker, versioned schema) exist specifically so that Option A can be added later without reworking Phase 1 code.

8. Data Requirements

Data	Stored Where (Phase 1)	Notes
Shopping list items	Local device + synced across user's own devices	Includes item name, quantity, category
Purchase history (item, date, quantity)	Local device only	Not transmitted to server in Phase 1
Prediction model state (per-item intervals, weights)	Local device only	Derived from purchase history
User account/profile data	Server (standard account system)	Email, auth tokens — not purchase data

Data	Stored Where (Phase 1)	Notes
Product/category reference list	Bundled with app, updated via app updates	Used for autocomplete and categorization

9. Assumptions & Constraints

- Users need a minimum data collection period (estimated 2-4 weeks, or a minimum number of purchases per item) before predictions become reliable; UX must account for this cold-start gap in Phase 1 without server priors
- On-device computation is assumed sufficient for the statistical models required (rolling averages, exponential smoothing); no heavy ML/deep learning is required for Phase 1
- Cross-platform framework choice (native vs. React Native/Flutter) is to be finalized separately but must support on-device ML execution (TensorFlow Lite has bindings for both major cross-platform frameworks)
- Server infrastructure for Phase 2 is not being built or provisioned as part of this phase

10. Success Metrics (Phase 1)

Metric	Target (example — to be finalized with stakeholders)
% of users still active after 4 weeks	TBD
% of suggestions accepted (vs. rejected/snoozed)	TBD
Average time-to-first-reliable-prediction per user	TBD
App store rating	TBD
Crash-free session rate	≥ 99%

11. Risks & Mitigations

Risk	Impact	Mitigation
Cold-start period feels "dumb" to new users, hurting early retention	High	Clear UI messaging during data collection; consider manual frequency hints as a stopgap until Phase 2 priors exist
On-device model accuracy is lower than a server-trained equivalent	Medium	Acceptable trade-off for Phase 1 privacy/offline goals; revisit if user feedback shows suggestion quality is a blocker
Architecture isn't actually extensible when Phase 2 begins	Medium	Enforce NFR-6 and the pluggable suggestion-ranker pattern in code review from day one
Users assume the app has cross-user "smart" features it doesn't have yet	Low	Marketing/onboarding copy should accurately reflect Phase 1 capabilities

12. Future Roadmap (Beyond Phase 1)

1. **Phase 2:** Introduce server-side cold-start priors (Option A) to improve new-user experience
2. **Phase 3:** Add receipt OCR, analytics dashboards, and delivery integrations
3. **Phase 4 (conditional):** Evaluate federated learning (Option C) only if simpler server-side approaches prove insufficient

End of Document